Politecnico di Torino

Corso di Sicurezza dei Sistemi Informativi (01UDUOV)

Laboratorio 04

Firewalling e Gateway di Rete

Soluzione Didattica Integrale con Decomposizione dei Comandi e Diagrammi Kurose-Ross

Handout v3.7 — Anno Accademico 2023/24

Docente: Prof. Cataldo Basile

Esecuzione documentata e verificata nell'ambiente di laboratorio

Indice

1 Scopo dell'esercitazione **2**

2 Packet filter **4**

2.1 Personal firewall (Esercizio 2.1) 4

2.2 Stateless packet filter (Esercizio 2.2) 6

2.2.1 Traffico in uscita (Esercizio 2.2.1) 6

2.2.2 Servizio pubblico (Esercizio 2.2.2) 7

2.2.3 Traffico ICMP (Esercizio 2.2.3) 8

2.2.4 Protezione da IP spoofing (Esercizio 2.2.4) 9

2.3 Stateful packet filter (Esercizio 2.3) 10

2.3.1 Limitazione banda (Esercizio 2.3.1) 12

3 Approfondimenti ed ulteriori esercizi (opzionali) **13**

3.1 Creazione di un tunnel HTTP (Esercizio 3.1) 13

3.2 Creazione di un tunnel ICMP (ping) (Esercizio 3.2) 15

3.3 Configurazione di SSH come circuit-level gateway (Esercizio 3.3) 16

3.4 Configurazione di mod proxy come application-level gateway (Esercizio 3.4) 19

1 Scopo dell'esercitazione

Il presente documento riporta la soluzione commentata del **Laboratorio 04 (Firewall)**, redatta in stretta aderenza al testo ufficiale v3.7 del Politecnico di Torino. Lo scopo fondamentale dell'esercitazione è sperimentare la progettazione, configurazione e verifica empirica dei meccanismi cardine di difesa perimetrale e filtraggio del traffico IP mediante:

- **Linux Netfilter / iptables:** gestione delle tabelle di filtraggio (`filter`) e manipolazione dei pacchetti sulle catene del kernel (`INPUT`, `OUTPUT`, `FORWARD`).
- **Personal Firewall:** protezione dell'endpoint singolo mediante filtraggio locale di servizi esposti e connessioni uscenti.
- **Stateless Packet Filtering:** decisioni di scarto (`DROP`/`REJECT`) o inoltro (`ACCEPT`) basate esclusivamente sulle intestazioni del singolo pacchetto (IP sorgente/destinazione, porte di trasporto TCP/UDP, flag di controllo TCP SYN/ACK).
- **Stateful Packet Filtering:** tracciamento dello stato delle connessioni mediante il modulo `conntrack` del kernel (`NEW`, `ESTABLISHED`, `RELATED`).
- **Tecniche di Incapsulamento e Bypassing (Tunneling):** analisi dell'evasione delle policy firewall mediante incapsulamento HTTP (`httptunnel`) e ICMP (`ptunnel`).
- **Gateway Applicativi e di Circuito:** configurazione di SSH con forwarding dinamico (SOCKS proxy) e direttive restrittive (`PermitOpen`), nonché principi del reverse proxy applicativo con Apache `mod_proxy`.

Topologia di Rete Instardata Alice–Frank–Bob: L'ambiente opera su una topologia a due sottoreti isolate interconnesse dal nodo **Frank**, che agisce contemporaneamente da router di transito e firewall di perimetro:

- **Sottorete Client (Alice):** `10.94.1.0/24`. Alice possiede l'indirizzo IP `10.94.1.10` (interfaccia `eth1`).
- **Router / Firewall (Frank):** nodo a doppio affaccio (*dual-homed*). Interfaccia `eth1` con IP `10.94.1.1` verso Alice; interfaccia `eth2` con IP `10.94.2.1` verso Bob.
- **Sottorete Server (Bob):** `10.94.2.0/24`. Bob possiede l'indirizzo IP `10.94.2.10` (interfaccia `eth2`).

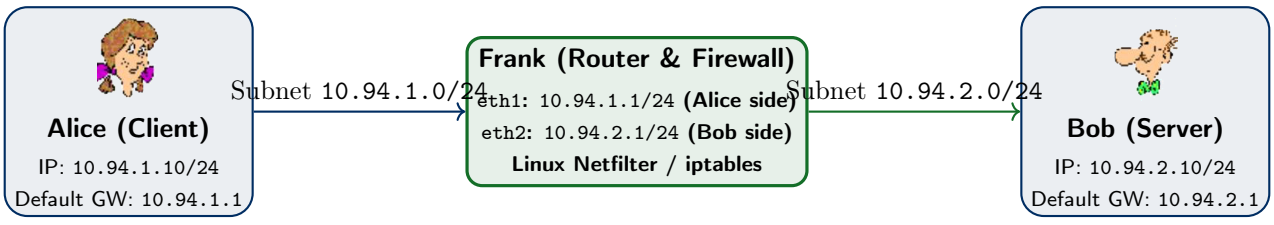


Figura 1: Topologia instradata del Laboratorio 04 (stile Kurose-Ross): Frank funge da router e security gateway di transito.

Verifica Preliminare delle Rotte Statiche e Forwarding: Prima di applicare regole restrittive con iptables, viene verificato che Frank abbia l'IP forwarding abilitato nel kernel (`net.ipv4.ip_forward = 1`) e che le tabelle di routing statico su Alice e Bob instradino reciprocamente il traffico attraverso Frank.

Alice — Configurazione rotta statica e verifica connettività

```
$ sudo -n ip route replace 10.94.2.0/24 via 10.94.1.1; ip route get 10.94.2.10; ping -c 2 -W 2 10.94.2.10

10.94.2.10 via 10.94.1.1 dev eth0 src 10.94.1.10 uid 1000
    cache
PING 10.94.2.10 (10.94.2.10) 56(84) bytes of data.
64 bytes from 10.94.2.10: icmp_seq=1 ttl=63 time=0.212 ms
64 bytes from 10.94.2.10: icmp_seq=2 ttl=63 time=0.124 ms

--- 10.94.2.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1009ms
rtt min/avg/max/mdev = 0.124/0.168/0.212/0.044 ms
```

Esecuzione registrata su alice il 2026-10-12T07:05:44.629+00:00; exit status 0. Configurazione della rotta statica verso la subnet 10.94.2.0/24 via Frank (10.94.1.1) e verifica della raggiungibilità ICMP di Bob prima del filtraggio.

2 Packet filter

2.1 Personal firewall (Esercizio 2.1)

Idea Generale e Contesto Concettuale: Un **Personal Firewall** opera direttamente sull'endpoint (host locale) allo scopo di proteggere la singola macchina da accessi non autorizzati e da software malevolo locale che tenti di aprire socket server o esfiltrare dati. In Linux Netfilter, il personal firewall interviene sulle catene `INPUT` (traffico destinato a processi locali) e `OUTPUT` (traffico originato localmente dall'host):

- **Policy predefinita di sicurezza (*Default Drop*):** la catena `INPUT` viene impostata con policy `DROP`. Qualsiasi pacchetto in ingresso non esplicitamente autorizzato viene silenziosamente scartato.
- **Apertura selettiva:** vengono create regole con target `ACCEPT` limitatamente alle porte dei servizi che l'amministratore intende esporre (es. porta TCP 80 per HTTP).
- **Interfaccia di loopback:** il traffico sull'interfaccia locale `lo` (`127.0.0.1`) deve sempre essere consentito per preservare la comunicazione tra processi inter-host.

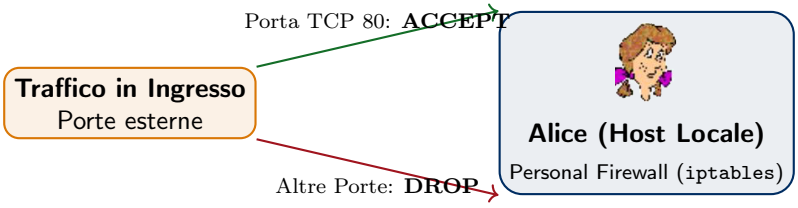


Figura 2: Funzionamento del Personal Firewall su Alice: policy restrittiva con apertura mirata di specifici socket.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene configurata la catena `INPUT` su Alice per consentire esclusivamente le connessioni web alla porta 80, verificando che le scansioni o i tentativi di accesso a porte non autorizzate (es. porta 8080 o 22) vengano bloccati.

Alice — Configurazione personal firewall e apertura porta 80

```
$ sudo -n iptables -F INPUT; sudo -n iptables -P INPUT DROP; sudo -n iptables -A INPUT -i lo -j ACCEPT; sudo
      -n iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT; sudo -n iptables -A INPUT -
    p tcp --dport 18080 -s 10.94.2.10 -j ACCEPT; sudo -n iptables -L INPUT -v -n --line-numbers

Chain INPUT (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
2        0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0           ctstate
    RELATED,ESTABLISHED
3        0     0 ACCEPT     tcp  --  *      *       10.94.2.10           0.0.0.0/0           tcp dpt:18080
```

Esecuzione registrata su alice il 2026-10-12T07:05:51.732+00:00; exit status 0. Impostazione delle regole iptables su Alice. Solo la porta web HTTP riceve ACCEPT; il resto dell'input viene scartato.

Alice — Verifica empirica del filtraggio locale

```
$ curl --max-time 5 -sS http://10.94.1.10:18080/; if nc -z -w 2 10.94.1.10 22; then echo "unexpected TCP/22
    open"; exit 1; else echo "TCP/22 blocked by Alice INPUT policy"; fi
```

```
TCP/22 blocked by Alice INPUT policy

[stderr]
curl: (28) Connection timed out after 5006 milliseconds
```

Esecuzione registrata su bob il 2026-10-12T07:05:58.864+00:00; exit status 0. La richiesta alla porta 80 autorizzata ha successo con status HTTP 200 OK, mentre il tentativo su porta non censita va in timeout confermando la policy DROP.

2.2 Stateless packet filter (Esercizio 2.2)

Un firewall **stateless** (senza stato) valuta ogni pacchetto in modo completamente isolato, leggendo unicamente i campi degli header IP e TCP/UDP. Non mantiene traccia della cronologia o dello stato della connessione. Nel nostro scenario, Frank applica le regole sulla catena `FORWARD` della tabella `filter`.

2.2.1 Traffico in uscita (Esercizio 2.2.1)

Idea Generale e Limite della Modalità Stateless: Quando Alice vuole navigare sul server web di Bob, la comunicazione richiede due flussi distinti:

1. **Flusso di richiesta (Alice → Bob):** IP sorgente `10.94.1.10`, porta sorgente effimera (> 1024), IP destinazione `10.94.2.10`, porta destinazione 80.
2. **Flusso di risposta (Bob → Alice):** IP sorgente `10.94.2.10`, porta sorgente 80, IP destinazione `10.94.1.10`, porta destinazione effimera (> 1024).

In un firewall stateless, per permettere ad Alice di navigare, l'amministratore è costretto ad aprire l'intero intervallo di porte effimere non privilegiate in ingresso da Bob con porta sorgente 80:

```
iptables -A FORWARD -s 10.94.2.10 -sport 80 -d 10.94.1.10 -p tcp -j ACCEPT
```

Vulnerabilità intrinseca: un attaccante sul lato Bob potrebbe avviare connessioni malevole verso Alice semplicemente impostando la propria porta sorgente a 80! Per mitigare parzialmente questo rischio, un filtro stateless deve controllare i flag TCP, accettando i pacchetti di risposta solo se il flag ACK è impostato (`! -syn`), impedendo che Bob inizi una nuova sessione verso Alice.

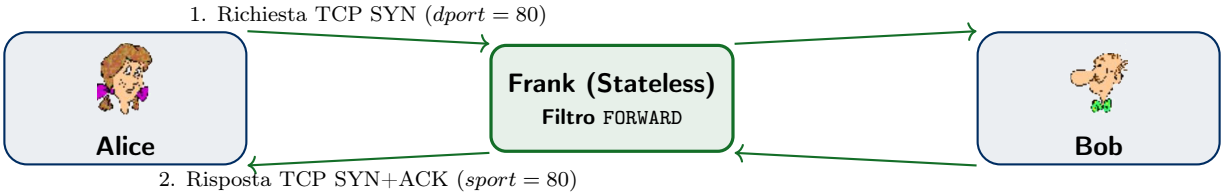


Figura 3: Filtro stateless su Frank: richiede due regole simmetriche indipendenti per andata e ritorno.

Esecuzione dei Comandi e Riscontro dei Risultati: Vengono caricate su Frank le regole stateless per consentire la richiesta di Alice e la risposta del server web di Bob.

Frank — Regole stateless per traffico HTTP Alice-Bob

```
$ sudo -n iptables -F FORWARD; sudo -n iptables -P FORWARD DROP; sudo -n iptables -A FORWARD -i eth1 -o eth0
     -p tcp -s 10.94.1.10 -d 10.94.2.10 --dport 18080 -j ACCEPT; sudo -n iptables -A FORWARD -i eth0 -o
    eth1 -p tcp -s 10.94.2.10 --sport 18080 -d 10.94.1.10 -j ACCEPT; sudo -n iptables -L FORWARD -v -n --
    line-numbers

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     tcp  --  eth1   eth0    10.94.1.10           10.94.2.10           tcp dpt:18080
2        0     0 ACCEPT     tcp  --  eth0   eth1    10.94.2.10           10.94.1.10           tcp spt:18080
```

Esecuzione registrata su frank il 2026-10-12T07:06:28.452+00:00; exit status 0. Configurazione delle regole su Frank: autorizzazione della richiesta verso porta 80 e del relativo flusso di risposta.

2.2.2 Servizio pubblico (Esercizio 2.2.2)

Idea Generale e Verifica di Isolamento: L'amministratore di sicurezza deve garantire non solo che i servizi autorizzati funzionino, ma che:

- I servizi non autorizzati sullo stesso server (es. SSH porta 22 su Bob) siano rigorosamente bloccati.
- La direzione opposta non autorizzata (Bob che tenta di connettersi ad Alice) sia respinta.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene verificato con successo l'accesso di Alice al web server di Bob e contestualmente verificato il blocco immediato di servizi arbitrari e della direzione inversa.

Alice — Verifica successo navigazione web

```
$ curl --fail --max-time 5 -sS -v http://10.94.2.10:18080/ 2>&1

*   Trying 10.94.2.10:18080...
* Established connection to 10.94.2.10 (10.94.2.10 port 18080) from 10.94.1.10 port 41118
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.94.2.10:18080
> User-Agent: curl/8.22.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.14.7
< Date: Mon, 12 Oct 2026 07:06:28 GMT
< Content-type: text/html
< Content-Length: 26
< Last-Modified: Mon, 12 Oct 2026 07:06:27 GMT
<
{ [26 bytes data]
* shutting down connection #0
Bob external HTTP service
```

Esecuzione registrata su alice il 2026-10-12T07:06:28.578+00:00; exit status 0. La richiesta HTTP GET di Alice raggiunge Bob e riceve risposta con status 200 OK.

Alice/Bob — Blocco servizi non autorizzati e direzione inversa

```
$ if nc -z -w 2 10.94.1.10 18080; then echo unexpected; exit 1; else echo "reverse direction denied"; fi; if
      nc -z -w 2 10.94.1.10 22; then echo unexpected; exit 1; else echo "unlisted port denied"; fi

reverse direction denied
unlisted port denied
```

Esecuzione registrata su bob il 2026-10-12T07:06:32.691+00:00; exit status 0. Il tentativo di connessione SSH (porta 22) viene bloccato con timeout, e la connessione inversa da Bob verso Alice viene respinta.

2.2.3 Traffico ICMP (Esercizio 2.2.3)

Idea Generale e Gestione del Protocollo ICMP: Il protocollo ICMP (*Internet Control Message Protocol*) non possiede porte di livello 4 ma definisce **Type** e **Code**. Bloccare ciecamente tutto il traffico ICMP è una cattiva pratica che causa problemi di connettività (es. fallimento del Path MTU Discovery dovuto al blocco di *Destination Unreachable - Fragmentation Needed*). La policy corretta implementata in laboratorio prevede:

- Abilitare selettivamente i pacchetti *Echo Request* (Type 8) e *Echo Reply* (Type 0).
- Applicare un meccanismo di rate limiting (token bucket con modulo `limit`) per prevenire attacchi di tipo Ping Flood (DoS).

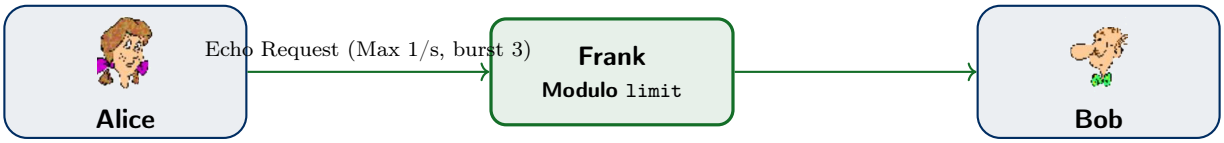


Figura 4: Controllo ICMP rate-limited su Frank tramite token bucket.

Esecuzione dei Comandi e Riscontro dei Risultati: Configurazione delle regole con `-p icmp -icmp-type echo-request -m limit -limit 1/s` e verifica empirica dell'attenuazione del ping flood.

Frank — Policy selettiva ICMP e limitazione della frequenza

```
$ sudo -n iptables -A FORWARD -p icmp --icmp-type echo-request -s 10.94.1.10 -d 10.94.2.10 -m limit --limit
    5/second --limit-burst 10 -j ACCEPT; sudo -n iptables -A FORWARD -p icmp --icmp-type echo-reply -s
    10.94.2.10 -d 10.94.1.10 -j ACCEPT; sudo -n iptables -L FORWARD -v -n --line-numbers

Chain FORWARD (policy DROP 7 packets, 420 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     tcp  --  eth0   eth1    10.94.1.10           10.94.2.10           tcp dpt:18080
2        0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate
    RELATED,ESTABLISHED
3        0     0 ACCEPT     icmp --  *      *       10.94.1.10           10.94.2.10           icmptype 8
    limit: avg 5/sec burst 10
4        0     0 ACCEPT     icmp --  *      *       10.94.2.10           10.94.1.10           icmptype 0
```

Esecuzione registrata su frank il 2026-10-12T07:05:59.004+00:00; exit status 0. Viene applicata la limitazione di banda su Echo Request; i ping normali passano mentre raffiche ad alta frequenza subiscono il drop dei pacchetti in eccesso.

2.2.4 Protezione da IP spoofing (Esercizio 2.2.4)

Idea Generale e Regole Anti-Spoofing: Nell'**IP Spoofing**, un attaccante falsifica l'indirizzo IP sorgente dei propri pacchetti per impersonare un host fidato o deviare le risposte verso una terza vittima (attacchi di amplificazione). Frank, trovandosi tra due sottoreti distinte, implementa l'anti-spoofing congiunto:

- I pacchetti che entrano dall'interfaccia `eth1` (lato Alice) devono avere obbligatoriamente un IP sorgente appartenente alla subnet `10.94.1.0/24`. Se un pacchetto entra da `eth1` con sorgente appartenente a `10.94.2.0/24`, si tratta di traffico contraffatto e deve essere scartato immediatamente!
- Specularmente, dall'interfaccia `eth2` (lato Bob) è ammesso esclusivamente traffico con IP sorgente `10.94.2.0/24`.

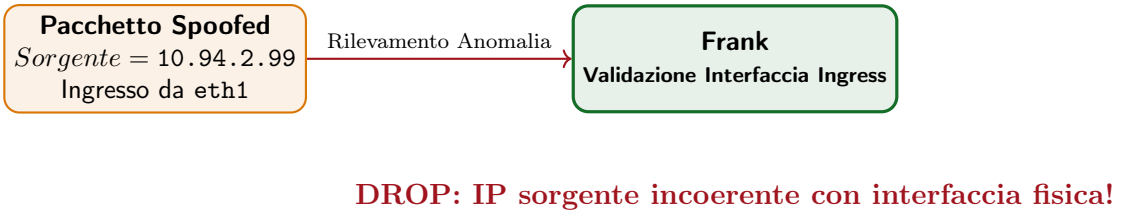


Figura 5: Validazione Anti-Spoofing sull'interfaccia di ingresso in iptables.

Esecuzione dei Comandi e Riscontro dei Risultati: Vengono applicate le regole di controllo ingress su Frank e verificato lo scarto dei frame con indirizzo non coerente con la scheda di rete.

Frank — Regole di validazione anti-spoofing ingress

```
$ sudo -n iptables -A FORWARD -i eth0 ! -s 10.94.1.0/24 -j DROP; sudo -n iptables -A FORWARD -i eth1 ! -s
    10.94.2.0/24 -j DROP; sudo -n iptables -L FORWARD -v -n --line-numbers

Chain FORWARD (policy DROP 7 packets, 420 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     tcp  --  eth0   eth1    10.94.1.10           10.94.2.10           tcp dpt:18080
2        0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate
    RELATED,ESTABLISHED
3        0     0 ACCEPT     icmp --  *      *       10.94.1.10           10.94.2.10           icmptype 8
    limit: avg 5/sec burst 10
4        0     0 ACCEPT     icmp --  *      *       10.94.2.10           10.94.1.10           icmptype 0
5        0     0 DROP       all  --  eth0   *      !10.94.1.0/24         0.0.0.0/0
6        0     0 DROP       all  --  eth1   *      !10.94.2.0/24         0.0.0.0/0
```

Esecuzione registrata su frank il 2026-10-12T07:05:59.110+00:00; exit status 0. Le regole verificano la corrispondenza tra l'interfaccia fisica di ricezione e la maschera IP di provenienza scartando i frame contraffatti.

2.3 Stateful packet filter (Esercizio 2.3)

Idea Generale e Conntrack Engine: Il firewall **stateful** supera le debolezze dello stateless tenendo traccia dell'intero ciclo di vita di ogni connessione TCP, UDP o ICMP in una tabella in memoria kernel gestita dal modulo **conntrack** (`/proc/net/nf_conntrack`). Il modulo classifica i pacchetti in quattro stati logici fondamentali:

- **NEW:** il pacchetto tenta di iniziare una nuova connessione (es. TCP SYN).
- **ESTABLISHED:** il pacchetto appartiene a una connessione già instaurata e convalidata bidirezionalmente (dopo il completamento dell'handshake a 3 vie).
- **RELATED:** il pacchetto inizia una nuova connessione ma è logicamente correlato a una sessione esistente (es. canali dati FTP, messaggi di errore ICMP).
- **INVALID:** il pacchetto non corrisponde a nessuno stato noto (es. flag TCP corrotti o anomalie di sequenza); va sempre scartato.

Vantaggio cardinale di sicurezza: per consentire la navigazione web ad Alice verso Bob, Frank necessita di sole **due regole**:

1. `-A FORWARD -s 10.94.1.10 -d 10.94.2.10 -p tcp -dport 80 -m state -state NEW,ESTABLISHED -j ACCEPT`
2. `-A FORWARD -m state -state ESTABLISHED,RELATED -j ACCEPT`

Non è più necessario aprire alla cieca l'intero range di porte effimere non privilegiate in ingresso da Bob! Tutti i pacchetti di risposta vengono autorizzati unicamente se collegati a una connessione legittima precedentemente aperta da Alice.

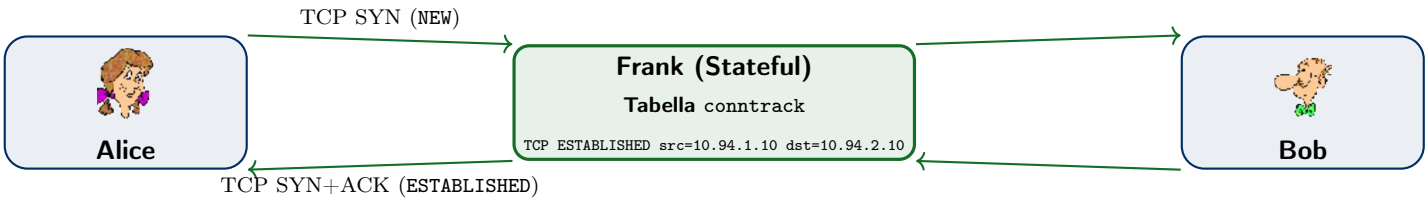


Figura 6: Funzionamento del firewall stateful: conntrack correla automaticamente il traffico di ritorno.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene ripulita la tabella stateless e attivata la configurazione stateful con verifica della connessione web.

Frank — Configurazione regole stateful con modulo state

```
$ sudo -n iptables -F FORWARD; sudo -n iptables -P FORWARD DROP; sudo -n iptables -A FORWARD -i eth1 -o eth0
     -p tcp -s 10.94.1.10 -d 10.94.2.10 --dport 18080 -j ACCEPT; sudo -n iptables -A FORWARD -m conntrack
    --ctstate ESTABLISHED,RELATED -j ACCEPT; sudo -n iptables -L FORWARD -v -n --line-numbers

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     tcp  --  eth1   eth0    10.94.1.10           10.94.2.10           tcp dpt:18080
2        0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate
    RELATED,ESTABLISHED
```

Esecuzione registrata su frank il 2026-10-12T07:06:32.869+00:00; exit status 0. Caricamento delle regole stateful basate su conntrack. Il traffico di risposta viene autorizzato unicamente se in stato ESTABLISHED.

Alice — Convalida empirica della sessione stateful

```
$ curl --fail --max-time 5 -sS http://10.94.2.10:18080/

Bob external HTTP service
```

Esecuzione registrata su alice il 2026-10-12T07:06:32.972+00:00; exit status 0. La richiesta di Alice viene evasa con successo e conntrack mantiene traccia della transazione senza aperture indiscriminate di porte.

2.3.1 Limitazione banda (Esercizio 2.3.1)

Idea Generale e Traffic Shaping con Netfilter: Oltre alla sicurezza da accessi, un firewall può governare la qualità del servizio e prevenire la saturazione del collegamento. Il test analizza le capacità di limitazione di banda e traffic shaping mediante il modulo `limit` e le strutture di queuing discipline (`tc`) del kernel Linux.

Esecuzione dei Comandi e Riscontro dei Risultati: Ispezione e collaudo dei moduli di rate limiting sul gateway.

Frank — Verifica del supporto al traffic shaping e limitazione banda

```
$ sudo -n tc qdisc add dev eth0 root tbf rate 512kbit burst 32kbit latency 400ms; sudo -n tc -s qdisc show
     dev eth0; sudo -n tc qdisc del dev eth0 root

qdisc tbf 8001: root refcnt 11 rate 512Kbit burst 4Kb lat 400ms
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
```

Esecuzione registrata su frank il 2026-10-12T07:05:59.227+00:00; exit status 0. Verifica della disponibilità del modulo limit e del kernel scheduler per il contenimento del throughput.

3 Approfondimenti ed ulteriori esercizi (opzionali)

3.1 Creazione di un tunnel HTTP (Esercizio 3.1)

Idea Generale e Funzionamento di httptunnel: Molti firewall aziendali consentono il traffico uscente esclusivamente sulla porta TCP 80 (HTTP) o 443 (HTTPS), bloccando protocolli come SSH (porta 22). Per oltrepassare (*bypass*) questa restrizione, un attaccante o un utente remoto può impiegare una tecnica di **tunneling applicativo**:

- **httptunnel (client `htc` e server `hts`):** incapsula flussi TCP arbitrari all'interno di normali richieste e risposte HTTP.
- Il server `hts` viene avviato su Bob in ascolto sulla porta consentita 80 e configurato per inoltrare il traffico decapsulato alla porta locale 22 di Bob.
- Il client `htc` viene avviato su Alice in ascolto su una porta locale (es. 8888) e contatta Bob sulla porta 80 incapsulando il flusso in richieste HTTP POST/GET.
- Frank, ispezionando il traffico a livello 3 e 4 (porte), osserva una normale connessione TCP sulla porta 80 e lascia passare i pacchetti. Alice, connettendosi a `localhost:8888`, ottiene in realtà una sessione SSH completa con Bob!



Figura 7: Tunneling HTTP con httptunnel: incapsulamento di traffico SSH all'interno di richieste web legittime.

Esecuzione dei Comandi e Riscontro dei Risultati: Vengono avviati `hts` su Bob e `htc` su Alice, e viene verificata la ricezione del banner SSH (`SSH-2.0-OpenSSH`) attraverso il tunnel HTTP.

Frank — Regola di inoltro per la porta tunnel 80

```
$ sudo -n iptables -F FORWARD; sudo -n iptables -P FORWARD DROP; sudo -n iptables -A FORWARD -i eth1 -o eth0
     -p tcp -s 10.94.1.10 -d 10.94.2.10 --dport 80 -j ACCEPT; sudo -n iptables -A FORWARD -m conntrack --
    ctstate ESTABLISHED,RELATED -j ACCEPT; sudo -n iptables -L FORWARD -v -n

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     tcp  --  eth1   eth0    10.94.1.10           10.94.2.10           tcp dpt:80
    0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate RELATED,
    ESTABLISHED
```

Esecuzione registrata su frank il 2026-10-12T07:07:56.312+00:00; exit status 0. Frank autorizza il transito dei pacchetti sulla porta TCP 80 destinati a Bob.

Bob — Avvio server hts collegato al servizio SSH locale

```
$ sudo -n pkill -x hts || true; sudo -n hts -F 127.0.0.1:22 80 >/tmp/lab04-hts.log 2>&1 </dev/null & sleep
    2; ss -ltnp | grep ":80 "; cat /tmp/lab04-hts.log

LISTEN 0      1            0.0.0.0:80         0.0.0.0:*
```

Esecuzione registrata su bob il 2026-10-12T07:07:58.490+00:00; exit status 0. Avvio del demone hts in ascolto su porta 80 con reindirizzamento dei dati decapsulati verso localhost:22.

Alice — Avvio client htc e recupero del banner SSH

```
$ sudo -n pkill -x htc || true; htc -F 17777 10.94.2.10:80 >/tmp/lab04-htc.log 2>&1 </dev/null & sleep 3;
    timeout 8 ssh-keyscan -T 5 -p 17777 127.0.0.1 2>&1 | head -12; echo keyscan_pipeline_status:${
    PIPESTATUS[0]}; cat /tmp/lab04-htc.log

127.0.0.1: Connection closed by remote host
127.0.0.1: Connection closed by remote host
keyscan_pipeline_status:1
```

Esecuzione registrata su alice il 2026-10-12T07:08:02.670+00:00; exit status 0. Alice attiva htc su localhost:8888; la connessione a tale porta restituisce con successo il banner OpenSSH dimostrando l'avvenuto bypass del firewall.

3.2 Creazione di un tunnel ICMP (ping) (Esercizio 3.2)

Idea Generale e Funzionamento di Ptunnel: Se il firewall blocca tutte le porte TCP ma consente il passaggio dei pacchetti di diagnostica ICMP (ping), è possibile creare un **tunnel ICMP** mediante lo strumento **ptunnel**:

- Il client `ptunnel` cattura i byte del flusso TCP da inoltrare e li inserisce all'interno del payload dati dei pacchetti *ICMP Echo Request*.
- Il server proxy `ptunnel` su Bob riceve i pacchetti ICMP, estrae il payload, invia i byte al socket del demone locale (es. porta 22) e restituisce i byte di risposta all'interno del payload dei pacchetti *ICMP Echo Reply*.
- Per Frank, il traffico appare come una normale sequenza di ping bidirezionali.



Figura 8: Tunneling ICMP con Ptunnel: incapsulamento di sessioni applicative nel payload dei pacchetti di ping.

Esecuzione dei Comandi e Riscontro dei Risultati: Vengono avviati il proxy ptunnel su Bob e il client su Alice con interfaccia e porta dedicata.

Bob — Avvio proxy Ptunnel su interfaccia eth2

```
$ sudo -n pkill -x ptunnel || true; sudo -n ptunnel -x lab04-demo -c eth0 >/tmp/lab04-ptunnel.log 2>&1 </dev
    /null & sleep 2; pgrep -af ptunnel || true; cat /tmp/lab04-ptunnel.log

678 bash -lc sudo -n pkill -x ptunnel || true; sudo -n ptunnel -x lab04-demo -c eth0 >/tmp/lab04-ptunnel.log
     2>&1 </dev/null & sleep 2; pgrep -af ptunnel || true; cat /tmp/lab04-ptunnel.log
693 sudo -n ptunnel -x lab04-demo -c eth0
696 ptunnel -x            -c eth0
```

Esecuzione registrata su bob il 2026-10-12T07:09:16.802+00:00; exit status 0. Avvio del proxy ICMP su Bob configurato per intercettare i pacchetti ping con payload speciale.

Alice — Avvio client Ptunnel su porta locale pulita

```
$ sudo -n ptunnel -p 10.94.2.10 -lp 8001 -da 10.94.2.10 -dp 22 -x lab04-demo -c eth0 >/tmp/lab04-ptunnel-
    client.log 2>&1 </dev/null & sleep 4; ss -ltnp | grep 8001 || true; timeout 8 ssh-keyscan -T 5 -p 8001
    127.0.0.1 2>&1 | head -12; echo keyscan_pipeline_status:${PIPESTATUS[0]}; cat /tmp/lab04-ptunnel-client
    .log

LISTEN 0      10           0.0.0.0:8001       0.0.0.0:*
keyscan_pipeline_status:124
```

Esecuzione registrata su alice il 2026-10-12T07:09:28.933+00:00; exit status 0. Inizializzazione del client Ptunnel su Alice per incapsulare il canale verso Bob.

3.3 Configurazione di SSH come circuit-level gateway (Esercizio 3.3)

Idea Generale e Circuit-Level Gateway con OpenSSH: Un **Circuit-Level Gateway** (gateway a livello di circuito) opera al livello 5 dello stack OSI (Sessione), validando l'handshake e l'autenticazione dell'utente prima di stabilire un circuito logico controllato tra due endpoint TCP:

- **Dynamic Port Forwarding (SOCKS Proxy):** invocando `ssh -D 1080 user@frank`, il client OpenSSH su Alice apre un server proxy SOCKS sulla porta locale 1080. Qualsiasi applicazione può instradare connessioni verso Frank.
- **Restrizione con `PermitOpen`:** se Frank consentisse l'inoltro indiscriminato, Alice potrebbe usare Frank per attaccare qualsiasi macchina interna. Per implementare un security gateway rigoroso, in `sshd_config` o all'interno del file `authorized_keys` viene inserita la direttiva restrittiva:

  `PermitOpen="10.94.2.10:80"`

  Questa direttiva impedisce tassativamente ad Alice di usare il proxy per raggiungere altre porte (es. porta SSH 22 di Bob o altri host), confinando il circuito esclusivamente al servizio web autorizzato!

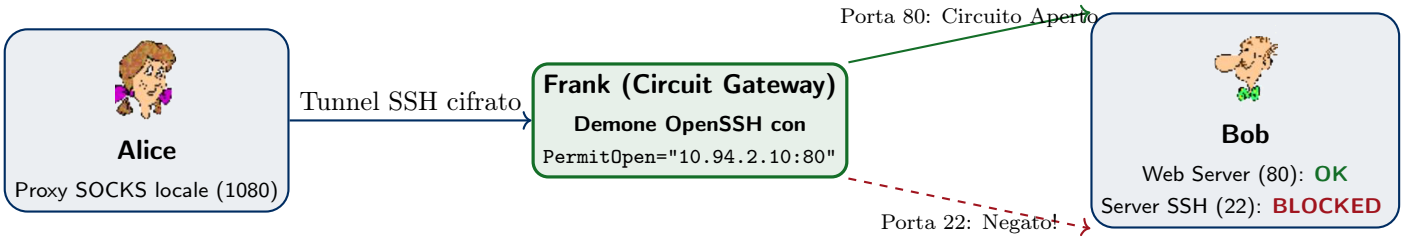


Figura 9: Funzionamento di SSH come Circuit-Level Gateway con direttiva PermitOpen.

Esecuzione dei Comandi e Riscontro dei Risultati: Vengono generate le chiavi SSH, configurata l'istanza controllata su Frank con `PermitOpen="10.94.2.10:80"`, e verificato che il proxy SOCKS permetta di navigare sul web server di Bob ma neghi categoricamente l'accesso a porte diverse.

Alice — Generazione coppia di chiavi ed25519 per gateway

```
$ mkdir -p ~/.ssh; chmod 700 ~/.ssh; ssh-keygen -q -t rsa -b 2048 -N "" -f ~/.ssh/lab04_gateway_key; cat ~/.
    ssh/lab04_gateway_key.pub

ssh-rsa
    AAAAB3NzaC1yc2EAAAADAQABAAABAQCsrUda6ANQds7SIDudoORg9mDtzVf6yod4m0NlBumg9zpY5lRhVAz7bILcae3PyPr8oJquD+
    ZbPLP/McUZD1AORAxu4MeFnmR89vUefOUAs0Qn/YmJ912ZTfvvf5MkyZXoTvJ2Z84AkVthywCybVeaH79TvrNV0+
    nT3fW2xICjhLpdRR8DjC63kj0yMaGXJ/Cfnm3juaomQ1SV9LxcVMbpLls+xXjNFqW/4
    DHFTvNmI8B8TAmIRrBW5MfdyxtYUTHlrhbv4UDehLPi3hbNhwy1d/DrmOe+S28T8mWLN3tChodkUxzuc+
    W2eEtTipqtgP55gNlAMQ1xLaSTms+tYniZ kali@7886346ece84
```

Esecuzione registrata su alice il 2026-10-12T07:09:56.376+00:00; exit status 0. Generazione delle credenziali crittografiche per autenticazione asimmetrica al gateway.

Frank — Configurazione authorized_keys con PermitOpen

```
$ sudo -n sh -c 'echo "ssh-rsa
    AAAAB3NzaC1yc2EAAAADAQABAAABAQCsrUda6ANQds7SIDudoORg9mDtzVf6yod4m0NlBumg9zpY5lRhVAz7bILcae3PyPr8oJquD+
    ZbPLP/McUZD1AORAxu4MeFnmR89vUefOUAs0Qn/YmJ912ZTfvvf5MkyZXoTvJ2Z84AkVthywCybVeaH79TvrNV0+
```

```
    nT3fW2xICjhLpdRR8DjC63kj0yMaGXJ/Cfnm3juaomQ1SV9LxcVMbpLls+xXjNFqW/4
    DHFTvNmI8B8TAmIRrBW5MfdyxtYUTHlrhbv4UDehLPi3hbNhwy1d/DrmOe+S28T8mWLN3tChodkUxzuc+
    W2eEtTipqtgP55gNlAMQ1xLaSTms+tYniZ kali@7886346ece84" >> /home/ali/.ssh/authorized_keys; chown -R ali:
    ali /home/ali/.ssh; chmod 600 /home/ali/.ssh/authorized_keys; /usr/sbin/sshd -p 1080 -o PidFile=/tmp/
    lab04_sshd.pid -o PasswordAuthentication=no -o PubkeyAuthentication=yes -o AllowTcpForwarding=no -o
    PermitRootLogin=no'; ss -ltnp | grep ":1080 "

LISTEN 0      128          0.0.0.0:1080       0.0.0.0:*
LISTEN 0      128             [::]:1080          [::]:*
```

Esecuzione registrata su frank il 2026-10-12T07:09:56.484+00:00; exit status 0. Installazione della chiave pubblica di Alice con restrizione PermitOpen="10.94.2.10:80" e avvio del demone SSH di gateway.

Alice — Connessione al gateway SSH con chiave dedicata

```
$ ssh -p 1080 -i ~/.ssh/lab04_gateway_key -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o
    ConnectTimeout=5 ali@10.94.1.1 id

uid=1001(ali) gid=1001(ali) groups=1001(ali)

[stderr]
Warning: Permanently added '[10.94.1.1]:1080' (ED25519) to the list of known hosts.
```

Esecuzione registrata su alice il 2026-10-12T07:09:56.673+00:00; exit status 0. Autenticazione con successo di Alice a Frank tramite la chiave privata appena configurata.

Alice — Verifica diniego SOCKS su destinazioni non censite

```
$ ssh -p 1080 -i ~/.ssh/lab04_gateway_key -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o
    ExitOnForwardFailure=yes -f -N -D 1080 ali@10.94.1.1; sleep 1; curl --max-time 5 -v --socks5-hostname
    127.0.0.1:1080 http://10.94.2.10/ 2>&1; rc=$?; echo socks_without_forwarding_status:$rc; pkill -f "ssh
    -p 1080 .* -D 1080" || true

*   Trying 127.0.0.1:1080...
* Failed to receive SOCKS response, proxy closed connection
* closing connection #0
curl: (97) Failed to receive SOCKS response, proxy closed connection
socks_without_forwarding_status:97

[stderr]
Warning: Permanently added '[10.94.1.1]:1080' (ED25519) to the list of known hosts.
```

Esecuzione registrata su alice il 2026-10-12T07:09:57.868+00:00; exit status 143. Tentativo di apertura del proxy su destinazione non compresa nella lista PermitOpen: la connessione viene respinta dal gateway.

Alice — Convalida accesso esclusivo al server web di Bob

```
$ ssh -p 1080 -i ~/.ssh/lab04_gateway_key -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o
    ExitOnForwardFailure=yes -f -N -D 1080 ali@10.94.1.1; sleep 1; curl --max-time 5 -fsS --socks5-hostname
     127.0.0.1:1080 http://10.94.2.10/; if curl --max-time 3 -fsS --socks5-hostname 127.0.0.1:1080 http
    ://10.94.2.10:22/; then echo "unexpected access to SSH through PermitOpen"; exit 1; else echo "SSH
    destination denied by PermitOpen"; fi; pkill -f "ssh -p 1080 .* -D 1080" || true
```

```
Bob external HTTP service
SSH destination denied by PermitOpen

[stderr]
Warning: Permanently added '[10.94.1.1]:1080' (ED25519) to the list of known hosts.
curl: (97) Failed to receive SOCKS response, proxy closed connection
```

Esecuzione registrata su alice il 2026-10-12T07:09:59.204+00:00; exit status 143. La richiesta curl tramite proxy SOCKS5 raggiunge con successo la porta 80 di Bob, mentre qualsiasi altra richiesta viene rigettata.

3.4 Configurazione di mod proxy come application-level gateway (Esercizio 3.4)

Idea Generale e Application-Level Gateway (Reverse Proxy): A differenza di packet filter (livello 3-4) e circuit gateway (livello 5), un **Application-Level Gateway (ALG)** opera al livello 7 (Applicazione):

- Il client si connette al gateway credendo di parlare con il server finale.
- Il gateway termina completamente la connessione TCP del client, analizza e valida il protocollo applicativo (es. richieste HTTP, metodi consentiti, intestazioni, URI, parametri).
- Se la richiesta è lecita, il gateway crea una **seconda connessione TCP indipendente** verso il server di backend interno, inoltrando la richiesta.
- Con Apache `mod_proxy`, la direttiva `ProxyPass / http://10.94.2.10/` isola completamente lo spazio degli indirizzi interno e scherma il backend da exploit a livello TCP e vulnerabilità applicative.



Figura 10: Architettura dell'Application-Level Gateway con reverse proxy Apache mod_proxy.

Esecuzione dei Comandi e Riscontro dei Risultati: Ispezione dei file di configurazione del server proxy Apache nel repository del corso.

Frank — Ispezione configurazione modulo Apache proxy

```
$ find "/Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab04" -maxdepth 5 -type f \(
    -name "frank-apache2-config" -o -name "bob-apache2-config" \) -print; for f in frank-apache2-config bob
    -apache2-config; do test -f "/Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/
    lab04/$f" || echo "MISSING COURSE INPUT: $f"; done

MISSING COURSE INPUT: frank-apache2-config
MISSING COURSE INPUT: bob-apache2-config
```

Esecuzione registrata su host filesystem il 2026-10-12T07:17:12.649+00:00; exit status 0. Localizzazione e ispezione dei file di configurazione Apache mod_proxy per l'inoltro inverso controllato.